

CLEYA.AI - COMPREHENSIVE FIXES PROMPT FOR REPLIT

MISSION

Implement all critical fixes identified in QA testing to make cleya.ai production-ready.

OVERVIEW

Platform: cleya.ai - AI-powered matchmaking for Indian startups **Current Status:** Conditional Pass
- Critical security issues **Goal:** Production-ready with 7.5+/10 matchmaking score

PRIORITY 1: CRITICAL SECURITY FIXES (P0)

1.1 Fix Environment Exposure

Issue: /api/health exposes env: "development" in production

Fix:

```
// In health check endpoint
const healthResponse = {
  status: 'ok',
  timestamp: new Date().toISOString(),
  // REMOVE: env: process.env.NODE_ENV,
  version: process.env.APP_VERSION || '1.0.0'
};
```

Files to modify:

- src/pages/api/health.ts or similar health endpoint
-

1.2 SQL Injection Prevention

Issue: Multiple SQL payloads cause 500 errors instead of proper rejection

Fix:

```
// Use parameterized queries EVERYWHERE
// BAD (vulnerable):
const query = `SELECT * FROM users WHERE email = '${email}'`;

// GOOD (safe):
const query = 'SELECT * FROM users WHERE email = $1';
const result = await db.query(query, [email]);

// Add input validation middleware
const validateInput = (req, res, next) => {
  const suspiciousPatterns = /["';--\|\|\|]|(\b(SELECT|INSERT|UPDATE|DELETE|DROP|UNION)\b)/i;
```

```

const checkObject = (obj) => {
  for (const key in obj) {
    if (typeof obj[key] === 'string' && suspiciousPatterns.test(obj[key])) {
      return res.status(400).json({ error: 'Invalid input detected' });
    }
  }
};

checkObject(req.body);
checkObject(req.query);
checkObject(req.params);
next();
};

```

Files to modify:

- All API routes in `src/pages/api/`
- Database query files
- Add middleware: `src/middleware/validateInput.ts`

1.3 XSS Prevention

Issue: Script tags and event handlers accepted in input fields

Fix:

```
npm install dompurify validator
```

```

// Create sanitization utility: src/utils/sanitize.js
import DOMPurify from 'isomorphic-dompurify';
import validator from 'validator';

export const sanitizeInput = (input) => {
  if (typeof input !== 'string') return input;

  // Remove all HTML tags
  let clean = DOMPurify.sanitize(input, {
    ALLOWED_TAGS: [],
    ALLOWED_ATTR: []
  });

  // Additional escaping
  clean = validator.escape(clean);

  // Trim whitespace
  clean = clean.trim();

  return clean;
};

```

```
export const sanitizeObject = (obj) => {
  const sanitized = {};
  for (const [key, value] of Object.entries(obj)) {
    sanitized[key] = sanitizeInput(value);
  }
  return sanitized;
};

// Apply to all user inputs
// Profile creation
app.post('/api/profile', (req, res) => {
  req.body = sanitizeObject(req.body);
  // ... rest of handler
});
```

Files to modify:

- Create: src/utils/sanitize.js
- All API routes that accept user input
- Profile creation/update endpoints

1.4 NoSQL Injection Prevention

Issue: MongoDB operators cause system errors

Fix:

```
// Add to validation middleware
const preventNoSQLInjection = (req, res, next) => {
  const hasMongoOperators = (obj) => {
    if (typeof obj !== 'object' || obj === null) return false;

    for (const key of Object.keys(obj)) {
      if (key.startsWith('$')) return true;
      if (hasMongoOperators(obj[key])) return true;
    }
    return false;
  };

  if (hasMongoOperators(req.body) || hasMongoOperators(req.query)) {
    return res.status(400).json({ error: 'Invalid operators detected' });
  }

  next();
};

// Use in all routes
app.use('/api/', preventNoSQLInjection);
```

1.5 Input Length Validation

Issue: 10,000+ character inputs accepted

Fix:

```
// Create validation schema: src/validation/schemas.js
import { z } from 'zod';

export const profileSchema = z.object({
  name: z.string().min(2).max(100),
  email: z.string().email().max(254),
  bio: z.string().max(1000).optional(),
  company: z.string().max(100).optional(),
  role: z.string().max(50),
  location: z.string().max(100),
  industry: z.string().max(50),
  interests: z.array(z.string().max(30)).max(10),
  linkedinUrl: z.string().url().max(200).optional(),
});

// Apply to routes
app.post('/api/profile', validateSchema(profileSchema), handler);
```

Install: npm install zod

1.6 Secure Admin Panel

Issue: /admin accessible without authentication

Fix:

```
// src/middleware/adminAuth.js
export const requireAdmin = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Authentication required' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    if (!decoded.isAdmin) {
      return res.status(403).json({ error: 'Admin access required' });
    }

    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({ error: 'Invalid token' });
  }
}
```

```
};

// Apply to admin routes
app.use('/admin', requireAdmin, adminRoutes);
```

1.7 Rate Limiting

Issue: No rate limiting allows bot attacks

Fix:

```
npm install express-rate-limit
```

```
// src/middleware/rateLimiter.js
import rateLimit from 'express-rate-limit';

// General API rate limit
export const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  message: 'Too many requests, please try again later',
  standardHeaders: true,
  legacyHeaders: false,
});

// Strict limit for auth endpoints
export const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5, // 5 attempts per 15 minutes
  skipSuccessfulRequests: true,
});

// Match generation limit
export const matchLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  max: 10, // 10 match requests per minute
});

// Apply in main app
app.use('/api/', apiLimiter);
app.use('/api/auth/', authLimiter);
app.use('/api/match/', matchLimiter);
```

1.8 CAPTCHA Implementation

Issue: No CAPTCHA on registration

Fix:

```
npm install react-google-recaptcha
```

```

// Frontend: Registration form
import ReCAPTCHA from 'react-google-recaptcha';

function RegistrationForm() {
  const [captchaToken, setCaptchaToken] = useState(null);

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!captchaToken) {
      alert('Please complete CAPTCHA');
      return;
    }

    const response = await fetch('/api/auth/register', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        ...formData,
        captchaToken
      })
    });
    // ...
  };

  return (
    <form onSubmit={handleSubmit}>
      { /* ... form fields ... */ }
      <ReCAPTCHA
        sitekey={process.env.NEXT_PUBLIC_RECAPTCHA_SITE_KEY}
        onChange={setCaptchaToken}
      />
      <button type="submit">Register</button>
    </form>
  );
}

// Backend: Verify CAPTCHA
// src/utils/verifyCaptcha.js
export const verifyCaptcha = async (token) => {
  const response = await fetch('https://www.google.com/recaptcha/api/siteverify', {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: `secret=${process.env.RECAPTCHA_SECRET_KEY}&response=${token}`
  });

  const data = await response.json();
  return data.success && data.score > 0.5;
};

```

1.9 Email Verification

Issue: No email verification allows fake accounts

Fix:

```
// src/utils/email.js
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: process.env.SMTP_PORT,
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASS
  }
});

export const sendVerificationEmail = async (email, token) => {
  const verificationUrl = `${process.env.APP_URL}/verify-email?token=${token}`;

  await transporter.sendMail({
    from: process.env.FROM_EMAIL,
    to: email,
    subject: 'Verify your Cleya account',
    html: `
      <h1>Welcome to Cleya!</h1>
      <p>Click the link below to verify your email:</p>
      <a href="${verificationUrl}">Verify Email</a>
      <p>This link expires in 24 hours.</p>
    `,
  });
};

// Registration endpoint
app.post('/api/auth/register', async (req, res) => {
  // ... create user ...

  const verificationToken = crypto.randomBytes(32).toString('hex');

  await db.query(
    'UPDATE users SET verification_token = $1, verified = false WHERE id = $2',
    [verificationToken, userId]
  );

  await sendVerificationEmail(email, verificationToken);

  res.json({
    message: 'Registration successful. Please check your email to verify your account.'
  });
});
```

```

});

// Verification endpoint
app.get('/api/auth/verify-email', async (req, res) => {
  const { token } = req.query;

  const result = await db.query(
    'UPDATE users SET verified = true, verification_token = NULL WHERE verification_token = $1 RETURNING'
    ↪ id',
    [token]
  );

  if (result.rowCount === 0) {
    return res.status(400).json({ error: 'Invalid or expired token' });
  }

  res.json({ message: 'Email verified successfully' });
});

```

PRIORITY 2: MATCHMAKING ALGORITHM FIXES

2.1 Implement Weighted Matching Algorithm

Issue: Keyword over-matching causes sector mismatches

Fix:

```

// src/utils/matchingAlgorithm.js

// Sector compatibility matrix
const SECTOR_COMPATIBILITY = {
  'Fintech': {
    'Fintech': 1.0,
    'SaaS': 0.7,
    'AI/ML': 0.6,
    'E-commerce': 0.5,
    'HealthTech': 0.3,
    'EdTech': 0.3,
    'CleanTech': 0.2,
    'default': 0.1
  },
  'HealthTech': {
    'HealthTech': 1.0,
    'AI/ML': 0.7,
    'SaaS': 0.5,
    'Fintech': 0.3,
    'default': 0.1
  },
  'SaaS': {
    'SaaS': 1.0,
    'Fintech': 0.7,

```



```

    'E-commerce': 0.6,
    'AI/ML': 0.6,
    'HealthTech': 0.5,
    'EdTech': 0.5,
    'default': 0.2
  },
  'AI/ML': {
    'AI/ML': 1.0,
    'SaaS': 0.6,
    'Fintech': 0.6,
    'HealthTech': 0.7,
    'EdTech': 0.5,
    'CleanTech': 0.4,
    'default': 0.2
  },
  // Add more sectors...
};

// Stage compatibility for founder-investor matching
const STAGE_COMPATIBILITY = {
  'pre-seed': {
    'angel': 1.0,
    'pre-seed': 1.0,
    'seed': 0.8,
    'early-stage': 0.7,
    'series-a': 0.3,
    'growth': 0.1,
    'default': 0.5
  },
  'seed': {
    'seed': 1.0,
    'angel': 0.9,
    'early-stage': 0.9,
    'series-a': 0.7,
    'pre-seed': 0.6,
    'growth': 0.3,
    'default': 0.5
  },
  'series-a': {
    'series-a': 1.0,
    'growth': 0.9,
    'early-stage': 0.7,
    'seed': 0.5,
    'venture': 0.8,
    'default': 0.5
  },
  // Add more stages...
};

// Main matching function
export const calculateMatchScore = (user1, user2) => {

```

```

// Determine match type
const isFounderInvestorMatch =
  (user1.segment === 'founder' && user2.segment === 'investor') ||
  (user1.segment === 'investor' && user2.segment === 'founder');

const isJobMatch =
  user1.segment === 'job_seeker' &&
  (user2.hiring || user2.segment === 'founder');

// Calculate component scores
const sectorScore = calculateSectorScore(user1.industry, user2.industry);
const intentScore = calculateIntentScore(user1.intent, user2.intent);
const stageScore = isFounderInvestorMatch
  ? calculateStageScore(user1.funding_stage, user2.check_size)
  : 0.5;
const locationScore = calculateLocationScore(user1.location, user2.location);
const qualityScore = calculateQualityScore(user1, user2);

// Weighted final score
let weights;
if (isFounderInvestorMatch) {
  weights = { sector: 0.35, intent: 0.25, stage: 0.20, location: 0.10, quality: 0.10 };
} else if (isJobMatch) {
  weights = { sector: 0.30, intent: 0.30, stage: 0.10, location: 0.15, quality: 0.15 };
} else {
  weights = { sector: 0.30, intent: 0.30, stage: 0.10, location: 0.15, quality: 0.15 };
}

const finalScore =
  (sectorScore * weights.sector) +
  (intentScore * weights.intent) +
  (stageScore * weights.stage) +
  (locationScore * weights.location) +
  (qualityScore * weights.quality);

return {
  score: Math.round(finalScore * 100),
  breakdown: {
    sector: Math.round(sectorScore * 100),
    intent: Math.round(intentScore * 100),
    stage: Math.round(stageScore * 100),
    location: Math.round(locationScore * 100),
    quality: Math.round(qualityScore * 100)
  },
  reasons: generateMatchReasons(user1, user2, sectorScore, intentScore, stageScore)
};
};

const calculateSectorScore = (industry1, industry2) => {
  if (!industry1 || !industry2) return 0.3;
  if (industry1 === industry2) return 1.0;

```

```

const compatibility = SECTOR_COMPATIBILITY[industry1]?.[industry2] ||
  SECTOR_COMPATIBILITY[industry2]?.[industry1] ||
  SECTOR_COMPATIBILITY[industry1]?.default || 0.1;

return compatibility;
};

const calculateIntentScore = (intent1, intent2) => {
  const complementaryPairs = [
    ['fundraising', 'investing'],
    ['hiring', 'job_seeking'],
    ['sales', 'seeking_services'],
    ['networking', 'networking']
  ];

  const intents1 = Array.isArray(intent1) ? intent1 : [intent1];
  const intents2 = Array.isArray(intent2) ? intent2 : [intent2];

  for (const intent of intents1) {
    for (const otherIntent of intents2) {
      const isMatch = complementaryPairs.some(pair =>
        (pair[0] === intent && pair[1] === otherIntent) ||
        (pair[0] === otherIntent && pair[1] === intent)
      );
      if (isMatch) return 1.0;
      if (intent === otherIntent) return 0.8;
    }
  }

  return 0.2;
};

const calculateStageScore = (fundingStage, checkSize) => {
  if (!fundingStage || !checkSize) return 0.5;

  const stageMap = {
    'pre-seed': 'pre-seed',
    'seed': 'seed',
    'series-a': 'series-a',
    'series-b': 'series-a',
    'bootstrapped': 'pre-seed'
  };

  const checkSizeMap = {
    '<$10K': 'angel',
    '$10K-$50K': 'angel',
    '$50K-$100K': 'pre-seed',
    '$100K-$500K': 'seed',
    '$500K+': 'series-a'
  };
};

```

```

const mappedStage = stageMap[fundingStage] || 'default';
const mappedCheckSize = checkSizeMap[checkSize] || 'default';

return STAGE_COMPATIBILITY[mappedStage]?.[mappedCheckSize] || 0.3;
};

const calculateLocationScore = (loc1, loc2) => {
  if (!loc1 || !loc2) return 0.5;

  const city1 = loc1.split(',')[0].trim();
  const city2 = loc2.split(',')[0].trim();

  if (city1 === city2) return 1.0;

  // Same country
  const country1 = loc1.split(',').pop().trim();
  const country2 = loc2.split(',').pop().trim();
  if (country1 === country2) return 0.7;

  // Same region (India, UAE, etc.)
  const regionMap = {
    'India': ['Bangalore', 'Mumbai', 'Delhi', 'Chennai', 'Hyderabad', 'Pune'],
    'UAE': ['Dubai', 'Abu Dhabi'],
    'USA': ['San Francisco', 'New York', 'Austin'],
    'Europe': ['London', 'Berlin', 'Amsterdam']
  };

  for (const [region, cities] of Object.entries(regionMap)) {
    if (cities.includes(city1) && cities.includes(city2)) return 0.8;
  }

  return 0.4;
};

const calculateQualityScore = (user1, user2) => {
  let score = 0.5;

  // Profile completeness
  if (user1.profile_completeness === 'complete') score += 0.2;
  else if (user1.profile_completeness === 'partial') score += 0.1;

  if (user2.profile_completeness === 'complete') score += 0.2;
  else if (user2.profile_completeness === 'partial') score += 0.1;

  // Behavior type
  if (user1.behavior_type === 'spammy' || user2.behavior_type === 'spammy') {
    score -= 0.3;
  }
  if (user1.behavior_type === 'premium' || user2.behavior_type === 'premium') {
    score += 0.1;
  }
};

```

```

    }

    return Math.min(Math.max(score, 0), 1);
  };

const generateMatchReasons = (user1, user2, sectorScore, intentScore, stageScore) => {
  const reasons = [];

  if (sectorScore >= 0.8) {
    reasons.push(`Both in ${user1.industry} || user2.industry sector`);
  } else if (sectorScore >= 0.5) {
    reasons.push(`Complementary sectors: ${user1.industry} & ${user2.industry}`);
  }

  if (intentScore >= 0.8) {
    reasons.push('Aligned goals');
  }

  if (stageScore >= 0.8) {
    reasons.push('Stage alignment');
  }

  return reasons;
};

```

2.2 Implement Minimum Match Threshold

Issue: Low-quality matches (33%) being shown to users

Fix:

```

// In match generation endpoint
export const generateMatches = async (userId, options = {}) => {
  const { limit = 10, minScore = 60 } = options; // Minimum 60% match

  const user = await getUserById(userId);
  const potentialMatches = await getPotentialMatches(user);

  const scoredMatches = potentialMatches.map(match => ({
    ...match,
    matchDetails: calculateMatchScore(user, match)
  }));

  // Filter by minimum score
  const qualifiedMatches = scoredMatches
    .filter(m => m.matchDetails.score >= minScore)
    .sort((a, b) => b.matchDetails.score - a.matchDetails.score)
    .slice(0, limit);

  // If not enough matches, use fallback

```

```

if (qualifiedMatches.length < 3) {
  const fallbackMatches = await getFallbackMatches(user, 3 - qualifiedMatches.length);
  qualifiedMatches.push(...fallbackMatches);
}

return qualifiedMatches;
};

// Fallback matching for cold start
const getFallbackMatches = async (user, count) => {
  // Get active, high-quality users in same location
  const fallback = await db.query(`
    SELECT * FROM users
    WHERE id != $1
      AND behavior_type != 'spammy'
      AND profile_completeness = 'complete'
      AND location ILIKE $2
    ORDER BY last_active DESC
    LIMIT $3
  `, [user.id, `%${user.location.split(',')[0]}%`, count]);

  return fallback.rows.map(match => ({
    ...match,
    matchDetails: {
      score: 50, // Base score for fallback
      breakdown: { fallback: 50 },
      reasons: ['Active in your area']
    },
    isFallback: true
  }));
};

```

2.3 Add Match Explanation

Issue: Users don't know why they were matched

Fix:

```

// Frontend: Match card component
function MatchCard({ match }) {
  const { score, breakdown, reasons } = match.matchDetails;

  return (
    <div className="match-card">
      <div className="match-header">
        <h3>{match.name}</h3>
        <div className="match-score">{score}% match</div>
      </div>

      { /* Match explanation */ }
    </div>
  );
}

```

```
<div className="match-reasons">
  <h4>Why this match?</h4>
  <ul>
    {reasons.map((reason, i) => (
      <li key={i}>{reason}</li>
    ))}
  </ul>
</div>

{/* Score breakdown (optional, for transparency) */}
<div className="score-breakdown">
  <div>Sector fit: {breakdown.sector}%</div>
  <div>Intent alignment: {breakdown.intent}%</div>
  <div>Stage match: {breakdown.stage}%</div>
  <div>Location: {breakdown.location}%</div>
</div>

<div className="match-actions">
  <button onClick={() => acceptMatch(match.id)}>Accept</button>
  <button onClick={() => declineMatch(match.id)}>Decline</button>
</div>
</div>
);
}
```

⚡ PRIORITY 3: PERFORMANCE OPTIMIZATIONS

3.1 API Response Time Optimization

Issue: 1243ms average (target <500ms)

Fix:

```
// 1. Add database indexing
// migration file
CREATE INDEX CONCURRENTLY idx_users_segment ON users(segment);
CREATE INDEX CONCURRENTLY idx_users_industry ON users(industry);
CREATE INDEX CONCURRENTLY idx_users_location ON users(location);
CREATE INDEX CONCURRENTLY idx_users_funding_stage ON users(funding_stage);
CREATE INDEX CONCURRENTLY idx_matches_user_id ON matches(user_id);
CREATE INDEX CONCURRENTLY idx_matches_status ON matches(status);

// 2. Implement caching
npm install ioredis

// src/utils/cache.js
import Redis from 'ioredis';

const redis = new Redis(process.env.REDIS_URL);

export const cacheMiddleware = (duration = 300) => {
```

```

return async (req, res, next) => {
  const key = `cache:${req.originalUrl}`;

  const cached = await redis.get(key);
  if (cached) {
    return res.json(JSON.parse(cached));
  }

  res.sendResponse = res.json;
  res.json = (body) => {
    redis.setex(key, duration, JSON.stringify(body));
    res.sendResponse(body);
  };

  next();
};

// Apply to match endpoint
app.get('/api/matches', cacheMiddleware(60), getMatches);

// 3. Optimize match query
const getOptimizedMatches = async (userId) => {
  // Use materialized view or pre-computed matches
  const matches = await db.query(`
    SELECT u.*, m.score, m.reasons
    FROM precomputed_matches m
    JOIN users u ON m.matched_user_id = u.id
    WHERE m.user_id = $1
    AND m.score >= 60
    ORDER BY m.score DESC
    LIMIT 10
  `, [userId]);

  return matches.rows;
};

// 4. Background match pre-computation
// Run every hour
const precomputeMatches = async () => {
  const users = await db.query('SELECT id FROM users WHERE active = true');

  for (const user of users.rows) {
    const matches = await generateMatches(user.id, { limit: 50 });

    // Store in precomputed_matches table
    for (const match of matches) {
      await db.query(`
        INSERT INTO precomputed_matches (user_id, matched_user_id, score, reasons, computed_at)
        VALUES ($1, $2, $3, $4, NOW())
        ON CONFLICT (user_id, matched_user_id)

```



```

        DO UPDATE SET score = $3, reasons = $4, computed_at = NOW()
        `, [user.id, match.id, match.matchDetails.score, JSON.stringify(match.matchDetails.reasons)]);
    }
}
};

// Schedule: node-cron or similar

```

3.2 Homepage Load Optimization

Issue: Homepage is slowest at 1774ms

Fix:

```

// 1. Lazy load components
import dynamic from 'next/dynamic';

const HeavyComponent = dynamic(() => import('../components/HeavyComponent'), {
  loading: () => <p>Loading...</p>,
  ssr: false // Disable server-side rendering if not needed
});

// 2. Optimize images
// Use Next.js Image component with proper sizing
import Image from 'next/image';

<Image
  src="/hero.jpg"
  alt="Hero"
  width={1200}
  height={600}
  priority // For above-fold images
  placeholder="blur"
  blurDataURL="data:image/jpeg;base64,..."
/>

// 3. Code splitting
// Use dynamic imports for routes
const Dashboard = dynamic(() => import('../pages/dashboard'));

// 4. Minimize JavaScript bundle
// Analyze: npm run analyze
// Use tree-shaking, remove unused dependencies

// 5. Add service worker for caching
// next-pwa or custom service worker

```

🚩 PRIORITY 4: UX IMPROVEMENTS

4.1 Streamline Onboarding

Issue: 5 steps, 5-7 minutes (target <3 minutes)

Fix:

```
// Reduce from 5 to 3 steps
const ONBOARDING_STEPS = [
  {
    id: 'basics',
    title: 'Tell us about yourself',
    fields: ['name', 'role', 'location']
  },
  {
    id: 'professional',
    title: 'Your professional details',
    fields: ['company', 'industry', 'experience_years', 'linkedin_url']
  },
  {
    id: 'intent',
    title: 'What brings you here?',
    fields: ['intent', 'interests']
  }
];

// Pre-fill from LinkedIn if available
const prefillFromLinkedIn = (linkedinData) => {
  return {
    name: linkedinData.name,
    role: linkedinData.title,
    company: linkedinData.company,
    location: linkedinData.location,
    industry: mapLinkedInIndustry(linkedinData.industry),
    linkedin_url: linkedinData.url
  };
};

// Add progress indicator with time estimate
function OnboardingProgress({ currentStep, totalSteps }) {
  const progress = (currentStep / totalSteps) * 100;
  const timeRemaining = (totalSteps - currentStep) * 45; // ~45s per step

  return (
    <div className="progress-container">
      <div className="progress-bar" style={{ width: `${progress}%` }} />
      <span>{timeRemaining}s remaining</span>
    </div>
  );
}
```

4.2 Fix Homepage Chatbot Buttons

Issue: Non-functional buttons in demo

Fix:

```
// Make demo buttons interactive
function HeroChatbot() {
  const [demoState, setDemoState] = useState('initial');

  const handleDemoClick = (scenario) => {
    setDemoState(scenario);

    // Simulate AI response
    setTimeout(() => {
      setDemoState('response');
    }, 800);
  };

  return (
    <div className="chatbot-demo">
      {demoState === 'initial' && (
        <>
          <p>Hey! I'm Cleya. What brings you here today?</p>
          <button onClick={() => handleDemoClick('fundraising')}>
            I'm raising a seed round
          </button>
          <button onClick={() => handleDemoClick('hiring')}>
            I'm looking to hire
          </button>
          <button onClick={() => handleDemoClick('investing')}>
            I'm looking for deals
          </button>
        </>
      )}

      {demoState === 'fundraising' && (
        <>
          <p>I found 3 investors that match your profile perfectly...</p>
          { /* Show investor cards */ }
        </>
      )}

      { /* Add reset button */ }
      <button onClick={() => setDemoState('initial')}>
        Try another scenario
      </button>
    </div>
  );
}
```

4.3 Fix Pricing Consistency

Issue: Pricing inconsistent between FAQ and Pricing page

Fix:

```
// Create single source of truth
export const PRICING_TIERS = [
  {
    name: 'Free',
    price: 0,
    priceDisplay: '₹0',
    period: 'forever',
    description: 'For individuals exploring the network',
    features: [
      '3 matches per week',
      'Basic profile',
      'Email notifications',
      'Community access'
    ],
    cta: 'Get Started',
    highlighted: false
  },
  {
    name: 'Pro',
    price: 999,
    priceDisplay: '₹999',
    period: 'month',
    description: 'For active founders and professionals',
    features: [
      'Unlimited matches',
      'Priority matching',
      'In-app messaging',
      'Deal Room access',
      'Analytics dashboard'
    ],
    cta: 'Start Pro Trial',
    highlighted: true
  },
  {
    name: 'Growth',
    price: 2999,
    priceDisplay: '₹2,999',
    period: 'month',
    description: 'For teams and power users',
    features: [
      'Everything in Pro',
      'Team accounts (up to 5)',
      'API access',
      'Dedicated support',
      'Custom integrations'
    ],
  },
]
```

```

    cta: 'Contact Sales',
    highlighted: false
  }
];

// Use in both Pricing page and FAQ
import { PRICING_TIERS } from '../config/pricing';

function PricingPage() {
  return (
    <div className="pricing-grid">
      {PRICING_TIERS.map(tier => (
        <PricingCard key={tier.name} tier={tier} />
      ))}
    </div>
  );
}

function FAQPage() {
  return (
    <div className="faq">
      <h3>What features are included in each plan?</h3>
      {PRICING_TIERS.map(tier => (
        <div key={tier.name}>
          <h4>{tier.name} ({tier.priceDisplay}/{tier.period})</h4>
          <ul>
            {tier.features.map(f => <li key={f}>{f}</li>)}
          </ul>
        </div>
      ))}
    </div>
  );
}

```

PRIORITY 5: TESTING & VALIDATION

5.1 Add Automated Tests

```
npm install --save-dev jest supertest @testing-library/react
```

```

// __tests__/matching.test.js
import { calculateMatchScore } from '../src/utils/matchingAlgorithm';

describe('Matchmaking Algorithm', () => {
  test('should give high score for same sector', () => {
    const user1 = { industry: 'Fintech', segment: 'founder', intent: ['fundraising'] };
    const user2 = { industry: 'Fintech', segment: 'investor', intent: ['investing'] };

    const result = calculateMatchScore(user1, user2);
    expect(result.score).toBeGreaterThan(80);
  });
});

```

```

});

test('should give low score for mismatched sectors', () => {
  const user1 = { industry: 'Fintech', segment: 'founder', intent: ['fundraising'] };
  const user2 = { industry: 'HealthTech', segment: 'investor', intent: ['investing'] };

  const result = calculateMatchScore(user1, user2);
  expect(result.score).toBeLessThan(50);
});

test('should not match spammy users', () => {
  const user1 = { industry: 'Fintech', behavior_type: 'active' };
  const user2 = { industry: 'Fintech', behavior_type: 'spammy' };

  const result = calculateMatchScore(user1, user2);
  expect(result.score).toBeLessThan(50);
});
});

// __tests__/security.test.js
import request from 'supertest';
import app from '../src/app';

describe('Security', () => {
  test('should reject SQL injection attempts', async () => {
    const response = await request(app)
      .post('/api/profile')
      .send({ name: "'; DROP TABLE users; --" });

    expect(response.status).toBe(400);
  });

  test('should reject XSS attempts', async () => {
    const response = await request(app)
      .post('/api/profile')
      .send({ bio: '<script>alert("xss")</script>' });

    expect(response.body.bio).not.toContain('<script>');
  });

  test('should enforce rate limiting', async () => {
    // Make 6 requests (limit is 5)
    for (let i = 0; i < 6; i++) {
      await request(app).post('/api/auth/login').send({});
    }

    const response = await request(app).post('/api/auth/login').send({});
    expect(response.status).toBe(429); // Too many requests
  });
});

```

5.2 Add Monitoring

```
// src/utils/monitoring.js
import { StatsD } from 'node-statsd';

const client = new StatsD({
  host: process.env.STATSD_HOST || 'localhost',
  port: process.env.STATSD_PORT || 8125,
  prefix: 'cleya.'
});

export const trackMetric = (name, value, tags = {}) => {
  const tagString = Object.entries(tags)
    .map(([k, v]) => `${k}:${v}`)
    .join(',');

  client.gauge(name, value, tagString);
};

export const trackTiming = (name, duration, tags = {}) => {
  const tagString = Object.entries(tags)
    .map(([k, v]) => `${k}:${v}`)
    .join(',');

  client.timing(name, duration, tagString);
};

// Use in API endpoints
app.get('/api/matches', async (req, res) => {
  const start = Date.now();

  const matches = await generateMatches(req.user.id);

  trackTiming('api.matches.response_time', Date.now() - start);
  trackMetric('api.matches.count', matches.length);

  res.json(matches);
});
```

IMPLEMENTATION CHECKLIST

Week 1: Critical Security

- Fix environment exposure
- Implement SQL injection prevention
- Add XSS sanitization
- Add NoSQL injection prevention

- Add input length validation
- Secure admin panel
- Implement rate limiting
- Add CAPTCHA
- Add email verification

Week 2: Matchmaking Algorithm

- Implement weighted matching algorithm
- Create sector compatibility matrix
- Add stage compatibility logic
- Implement minimum match threshold (60%)
- Add fallback matching for cold start
- Add match explanations

Week 3: Performance & UX

- Add database indexes
- Implement Redis caching
- Create precomputed matches table
- Optimize homepage load
- Streamline onboarding (5→3 steps)
- Fix homepage chatbot buttons
- Fix pricing consistency

Week 4: Testing & Polish

- Add automated tests
- Add monitoring/alerting
- Security audit
- Performance testing
- Bug fixes

 SUCCESS CRITERIA

Before deploying to production, verify:

Criteria	Target	How to Test
Security score	9+/10	Run security scan
Match accuracy	7+/10	Manual QA testing
API response time	<500ms P95	Load testing
Page load time	<1000ms	Lighthouse audit
Test coverage	80%+	Jest coverage report
Onboarding completion	80%+	Analytics tracking

DEPLOYMENT NOTES

1. **Database Migrations:** Run during low-traffic period
2. **Feature Flags:** Use for gradual rollout
3. **Rollback Plan:** Keep previous version ready
4. **Monitoring:** Watch error rates post-deploy

Estimated Timeline: 4 weeks **Team Required:** 2-3 developers **Priority Order:** Security → Match-making → Performance → UX

Good luck! 